



Google Cloud

Laboratorio Práctico

Este documento es una guía para desarrollar un **taller práctico** que consta en realizar el despliegue profesional de aplicaciones web con Python (Django) en Google Cloud Platform.

BOOTCAMP

Desarrollo de Aplicaciones Web Full Stack Python

INSTRUCTOR

[Francisco Felipe Cuevas Cerón](#)



AVISO IMPORTANTE

Toda la información técnica y procedimientos de este manual han sido extraídos directamente de la documentación oficial de Google Cloud:

<https://docs.cloud.google.com/python/django/run>

Introducción al Laboratorio

Bienvenido a este laboratorio práctico. Hasta ahora, tus aplicaciones Django han vivido en un entorno local, limitadas a tu propia máquina. El objetivo de este manual es profesionalizar tu flujo de trabajo, llevándote de un entorno de desarrollo a una arquitectura de producción real, segura y escalable en la nube.

Al finalizar estas etapas, no solo tendrás una aplicación pública, sino que habrás construido una infraestructura moderna que desacopla los servicios (base de datos, almacenamiento de archivos y ejecución de código), tal como se hace en la industria.



Ciberseguridad

Aprenderás a gestionar
secretos de ciberseguridad

Despliegues

Aprenderás a automatizar
despliegues de aplicaciones

Bases de Datos

Aprenderás a usar bases de
datos administradas

Etapa 1 - Activación de la Cuenta y Créditos Gratuitos



Objetivo de la Etapa: En esta etapa inicial, aseguraremos el acceso a los recursos de Google Cloud Platform (GCP) sin incurrir en costos, aprovechando el programa de prueba gratuita para nuevos usuarios.

Paso 1 - Prueba Gratuita

Accede a la consola de Google Cloud y activa la "Prueba Gratuita". Esto te otorgará \$300 USD en créditos válidos por 90 días.

Paso 2 - Facturación

Verifica que la facturación esté habilitada en tu proyecto. Aunque es gratuito, Google requiere una tarjeta para validar tu identidad. No se realizarán cobros automáticos al finalizar la prueba a menos que actualices manualmente tu cuenta.

Etapa 2 - Preparación del Entorno e Infraestructura

 **Objetivo de la Etapa:** En esta fase construiremos los cimientos de tu aplicación. Configuraremos tu computadora local para que pueda "hablar" con Google Cloud y habilitaremos los servicios (APIs) necesarios en la nube. Imagina que estamos conectando las tuberías de agua y electricidad antes de construir la casa: sin esto, nada funciona.

 **Advertencia:** Antes de empezar, debes elegir una región geográfica de Google Cloud (por ejemplo: **us-central1** en Iowa, EE.UU.). Para evitar errores de conexión y latencia, **TODOS** los recursos que crees en este laboratorio (Base de datos, Cloud Run, Almacenamiento) deben estar estrictamente en la **misma región**. ¡No las mezcles!

Paso 1 - Instalación de la "Torre de Control" (Google Cloud CLI)

- Necesitamos la herramienta de línea de comandos `gcloud` para enviar órdenes a la nube desde tu terminal.
 - Si no la tienes, instálala siguiendo las instrucciones oficiales para tu sistema operativo (Windows/Mac/Linux).
 - Abre tu terminal y ejecuta el siguiente comando para iniciar sesión y conectar tu cuenta:

```
gcloud init
```

- Sigue las instrucciones en pantalla para loguearte con tu correo de Gmail.

Paso 2 - Creación del Proyecto

- El "Proyecto" es el contenedor donde vivirán todos tus recursos.
 - En la consola web o durante el `gcloud init`, crea un nuevo proyecto (ej: `bootcamp-django-[tu-nombre]`).
 - Asegúrate de que este proyecto tenga la facturación (billing) vinculada a la cuenta de prueba de \$300 USD que activaste en la **Etapa 1**.

Paso 3 - Habilitación de Motores (APIs)

- Google Cloud tiene miles de servicios apagados por defecto. Debemos encender los que usaremos. Ve a la sección "APIs y servicios" -> "Biblioteca" en la consola, busca y **habilita** las siguientes APIs una por una:
 - **Cloud Run API:** Para ejecutar tus contenedores.
 - **Cloud SQL Admin API:** Para gestionar tu base de datos PostgreSQL.
 - **Cloud Build API:** Para automatizar el despliegue (DevOps).
 - **Secret Manager API:** Para guardar tus contraseñas de forma segura.
 - **Artifact Registry API:** Para almacenar las imágenes Docker de tu app.

Paso 4 - Verificación de tu Entorno Python

- Este laboratorio requiere Python 3.10 o superior para evitar conflictos con las versiones modernas de Django.
 - Verifica tu versión en la terminal:

```
python --version
```

- Si ves algo como Python 3.10.x o 3.11.x, estás listo.

Paso 5 - PRECAUCIÓN Sobre el Registro de Artefactos (Artifact Registry)

En las próximas etapas, tu código se empaquetará en una "imagen" (como un archivo zip inteligente) y se guardará en un repositorio llamado `cloud-run-source-deploy`. Generalmente, Google crea este repositorio automáticamente durante tu primer despliegue.

Sin embargo, para prevenir cualquier error de "Repositorio no encontrado" en el futuro, si deseas puedes crearlo manualmente ahora mismo con este comando (asegúrate de usar tu región elegida):

```
gcloud artifacts repositories create cloud-run-source-deploy \
  --repository-format=docker \
  --location=us-central1 \
  --description="Repositorio para Cloud Run"
```

Si la terminal te dice que "ya existe", ¡perfecto! Ya tienes todo listo.

Etapas 3 - Configuración de Base de Datos Profesional (Cloud SQL)

- Objetivo de la Etapa:** Es hora de abandonar **db.sqlite3**. SQLite es excelente para prototipos locales, pero en una aplicación profesional concurrente necesitamos un motor robusto. En esta etapa provisionaremos una instancia de **PostgreSQL** totalmente gestionada por Google. Configuraremos una versión "Sandbox" (Caja de arena) que es ideal para desarrollo porque ofrece la potencia de SQL Enterprise gratis (si te quedan créditos disponibles).

Paso 1 - Creación de la Instancia SQL

- En la consola de Google Cloud, busca y selecciona **Cloud SQL**.
- Haz clic en el botón **Crear Instancia**.
- Selecciona el motor **PostgreSQL** (*es el que mejor se lleva con Django*).
- **CONFIGURACIÓN CRÍTICA** - Para no gastar demás:
 - **ID de la instancia:** Ponle un nombre identificable (ej: `miapp-db-server`).
 - **Contraseña:** Escribe una contraseña fuerte para el usuario principal postgres y **anótala**.
 - **Edición de Cloud SQL:** Selecciona **Enterprise**.
 - **Preajuste (Preset):** Selecciona **Sandbox** (*Entorno de pruebas*). Esto eliminará las opciones costosas de alta disponibilidad que no necesitamos ahora.
 - **¡MUY IMPORTANTE! - Región:** Selecciona **exactly la misma región** que elegiste en la **Etapas 2** (ej: `us-central1`). Si las pones en zonas diferentes, la latencia hará que tu web sea lenta.
- Haz clic en **Crear Instancia** y espera unos minutos (*puede tardar unos 10 minutos en iniciarse*).

Paso 2 - Creación de la Base de Datos

- Una "Instancia" es como un servidor que puede contener muchas bases de datos. Ahora crearemos la específica para tu proyecto.
 - Una vez que la instancia esté activa (*check verde*), haz clic en su nombre para entrar a los detalles.
 - En el menú izquierdo, ve a la pestaña **Bases de datos**.
 - Haz clic en **Crear base de datos**.
 - Nombre: `miapp_db` (o el nombre que prefieras).
 - Haz clic en **Crear**.

Paso 3 - Creación de Usuario de Aplicación (Seguridad)

- Por seguridad, no es recomendable que tu aplicación Django se conecte usando el "Superusuario" (postgres). Vamos a crear un usuario con permisos normales.
 - En el menú izquierdo de tu instancia, ve a la pestaña **Usuarios**.
 - Haz clic en **Agregar cuenta de usuario**.
 - **Nombre de usuario:** `django_user` (o similar).
 - **Contraseña:** Crea una contraseña segura (diferente a la del superusuario) y **guárdala**, la usaremos en la **Etapas 5** para el archivo de secretos.
 - Haz clic en **Agregar**.

Etapla 4 - Almacenamiento de Archivos Estáticos (Cloud Storage)

- i** **Objetivo de la Etapa:** Los contenedores de Cloud Run son "efímeros", lo que significa que cada vez que despliegas o el sistema escala, el contenedor se destruye y se crea uno nuevo desde cero. Si guardas una foto de perfil o un archivo CSS dentro del contenedor, **desaparecerá**. Para solucionar esto, usaremos **Cloud Storage**, un servicio de almacenamiento de objetos (Buckets) que guardará tus archivos estáticos (CSS, JS) y multimedia (imágenes subidas por usuarios) de forma permanente y segura fuera del contenedor.

Paso 1 - Creación del Bucket

- En la consola de Google Cloud, ve al menú de navegación y selecciona **Cloud Storage > Buckets**.
- Haz clic en **Crear bucket**.

Paso 2 - Asignación de Nombre y Región

- **Nombre del bucket:** Debe ser único en todo el mundo (*como un dominio web*). Te recomendamos usar el patrón: `media-global-[tu-id-proyecto]`.
 - *Ejemplo:* Si tu proyecto se llama `bootcamp-felipe`, tu bucket podría ser `media-global-bootcamp-felipe`.
- **Ubicación:** Elige **Región** y selecciona **EXACTAMENTE LA MISMA** que usaste en las etapas anteriores (ej: `us-central1`). Esto evita costos de transferencia de datos y latencia.

Paso 3 - Configuración de Acceso Público (CRÍTICO)

- Por defecto, Google bloquea todo acceso público por seguridad. Pero como este bucket servirá tu CSS e imágenes a los navegadores de los usuarios, necesitamos abrirlo controladamente.
 - Baja hasta la sección de "Elegir cómo controlar el acceso a los objetos".
 - **Desmarca** la casilla que dice "Imponer prevención de acceso público" (*Enforce public access prevention*). Si la dejas marcada, nadie podrá ver tus imágenes.
 - En "Control de acceso", selecciona **Preciso** (*Fine-grained*). Esto nos permitirá más adelante usar código para decir qué archivos son públicos y cuáles privados.
 - Deja el resto de opciones por defecto y haz clic en **Crear**.

Paso 4 - Confirmación

- Es posible que Google te muestre una advertencia diciendo que el bucket será público.
- Confirma que entiendes el riesgo (*es intencional para este caso de uso*) y finaliza la creación.
- Ahora tienes un "disco duro" infinito en la nube. En la próxima etapa, le daremos las llaves a Django para que pueda guardar cosas ahí.

Etapa 5 - Gestión de Secretos y Ciberseguridad (Secret Manager)

Objetivo de la Etapa: Un error de novato es "quemar" contraseñas (hardcoding) dentro del código fuente (**settings.py**). Si subes eso a GitHub, los hackers tendrán acceso a tu base de datos en segundos. Por eso en esta etapa, guardaremos las credenciales en una bóveda digital encriptada llamada **Secret Manager**. Tu aplicación leerá estas credenciales "al vuelo" solo cuando las necesite.

Paso 1 - Preparación del Archivo de Secretos

- En la raíz de tu proyecto (donde está `manage.py`), crea un archivo llamado `.env`.
- Copia y pega el siguiente código, reemplaza los valores con **tus datos reales** de las etapas anteriores:

```
# Estructura de conexión:
postgres://USUARIO:PASSWORD@//cloudsql/PROYECTO:REGION:INSTANCIA/NOMBRE_DB
DATABASE_URL=postgres://django_user:tu_password_segura@//cloudsql/bootcamp-felipe:us-central1:miapp-db-server/miapp_db

# El nombre de tu bucket creado en la Etapa 4
GS_BUCKET_NAME=media-global-bootcamp-felipe

# Genera una clave aleatoria (puedes inventar una cadena larga de letras y números)
SECRET_KEY=k34j5h23k4jh5k23jh45kj2h34k5j2h34k5jh
```

- El formato de `DATABASE_URL` es especial para que la librería `django-environ` entienda que debe usar el socket de Cloud SQL.

Paso 2 - Creación del Secreto en la Nube

- En la consola, busca **Secret Manager**.
- Haz clic en **Crear Secreto**.
- **Nombre:** `django_settings` (Es vital que uses este nombre exacto, lo usaremos en el futuro).
- **Valor del secreto:** Copia y pega **todo** el contenido de tu archivo `.env` que creaste en el paso anterior.
- Deja el resto por defecto y haz clic en **Crear Secreto**.

Paso 3 - Autorización de Acceso (IAM)

Aquí es donde la magia ocurre. Por defecto, **nadie** (ni siquiera tu aplicación) puede leer este secreto. Debemos darle permiso explícito a la "identidad" que usará Cloud Run. Cloud Run usa por defecto la "Cuenta de servicio de Compute Engine". Vamos a darle la llave de la bóveda.

- Abre tu terminal (o *Cloud Shell*) y obtén el número de tu proyecto:

```
gcloud projects describe $(gcloud config get-value project) --format="value(projectNumber)"
```

- Anota el número que aparece, ej: 123456789
- Ejecuta el siguiente comando para darle permiso a tu cuenta de servicio para leer el secreto (**Reemplaza** NUMERO_PROYECTO **con el número que obtuviste arriba**):

```
gcloud secrets add-iam-policy-binding django_settings \  
--member serviceAccount:NUMERO_PROYECTO-compute@developer.gserviceaccount.com \  
--role roles/secretmanager.secretAccessor
```

- **Referencia técnica:** Este comando vincula el rol de "Accesor de Secretos" a la cuenta de servicio por defecto del proyecto.

Paso 4 - Secreto de Superusuario

Para poder entrar al panel de administración de Django (/admin) en producción, necesitaremos crear un superusuario automáticamente más adelante.

- Crea otro secreto nuevo en Secret Manager llamado: `superuser_password`.
- En el valor, escribe una contraseña segura que quieras usar para tu admin.
- Ejecuta el mismo comando de permisos del **Paso 3**, pero cambiando el nombre del secreto:

```
gcloud secrets add-iam-policy-binding superuser_password \  
--member serviceAccount:NUMERO_PROYECTO-compute@developer.gserviceaccount.com \  
--role roles/secretmanager.secretAccessor
```

- **¡Seguridad configurada!** Ahora tus contraseñas están en una caja fuerte y solo tu aplicación tiene la combinación.

Etapa 6 - Adaptación del Código Django

-  **Objetivo de la Etapa:** Vamos a realizar una "cirugía a corazón abierto" en tu proyecto. Modificaremos el archivo **settings.py** para que sea inteligente: si detecta que está en la nube, leerá los secretos de la bóveda, usará la base de datos PostgreSQL y guardará archivos en el Bucket. Si está en tu PC, seguirá funcionando normal. Al finalizar esta etapa, tu código será **"Cloud Native"**, es decir, estará listo para ser empaquetado y enviado a los servidores de Google

Paso 1 - Instalación de Dependencias

- Abre el archivo `requirements.txt` en la raíz de tu proyecto.
- Agrega las siguientes líneas al final (o crea el archivo si no existe):

```
psycopg2-binary>=2.9.0      # Conector para PostgreSQL
django-environ>=0.10.0      # Para leer variables de entorno y .env
google-cloud-secret-manager>=2.16.0  # Para leer secretos de la bóveda
djangdjango-storages[google]>=1.13.0  # Para guardar archivos en Cloud Storage
gunicorn>=20.1.0            # Servidor web de producción
```

- Instálalas en tu entorno virtual ejecutando el siguiente comando:

```
pip install -r requirements.txt.
```

Paso 2 - Importaciones y Configuración de Secretos

- Django necesita saber leer la "Bóveda" de Google antes de cargar cualquier otra cosa.
 - Abre `settings.py`.
 - Reemplaza las importaciones iniciales y la sección de `SECRET_KEY` con este bloque de código robusto (*revisa la siguiente página*):


```

import io
import os
from urllib.parse import urlparse
import environ
from google.cloud import secretmanager

# Construir rutas dentro del proyecto
BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))

# Inicializar django-environ
env = environ.Env(DEBUG=(bool, False))
env_file = os.path.join(BASE_DIR, ".env")

# LÓGICA DE CARGA DE SECRETOS (NUBE VS LOCAL)
if os.path.isfile(env_file):
    # Si existe .env local, úsalo (Desarrollo)
    env.read_env(env_file)
elif os.environ.get("GOOGLE_CLOUD_PROJECT", None):
    # Si estamos en Google Cloud, leer de Secret Manager
    project_id = os.environ.get("GOOGLE_CLOUD_PROJECT")
    client = secretmanager.SecretManagerServiceClient()
    settings_name = os.environ.get("SETTINGS_NAME", "django_settings")
    name = f"projects/{project_id}/secrets/{settings_name}/versions/latest"
    payload = client.access_secret_version(name=name).payload.data.decode("UTF-8")
    env.read_env(io.StringIO(payload))
else:
    raise Exception("No se detectó .env local ni proyecto de Google Cloud.")

SECRET_KEY = env("SECRET_KEY")
DEBUG = env("DEBUG", default=False)

```

Paso 3 - Configuración de Base de Datos y Seguridad Web

- Cloud Run necesita configuraciones especiales para saber en qué URL está viviendo y cómo conectarse a la SQL.
 - Busca la sección `DATABASES` y `ALLOWED_HOSTS` en tu `settings.py` y **reemplázalas** por esto (*revisa la siguiente página*):

```
# Configuración de Base de Datos (Lee la URL postgres:// desde los secretos)
DATABASES = {"default": env.db()}

# Seguridad y Dominios Permitidos (CSRF)
# Esto permite que Django confíe en la URL segura de Cloud Run
CLOUDRUN_SERVICE_URLS = env("CLOUDRUN_SERVICE_URLS", default=None)
if CLOUDRUN_SERVICE_URLS:
    CSRF_TRUSTED_ORIGINS = CLOUDRUN_SERVICE_URLS.split(",")
    ALLOWED_HOSTS = [urlparse(url).netloc for url in CSRF_TRUSTED_ORIGINS]
else:
    ALLOWED_HOSTS = ["*"] # Solo para desarrollo local

# Redirección a HTTPS (Seguridad obligatoria en producción)
SECURE_SSL_REDIRECT = False # Cambiar a True solo si tienes dominio personalizado, Cloud Run ya maneja HTTPS por defecto
SECURE_PROXY_SSL_HEADER = ("HTTP_X_FORWARDED_PROTO", "https")
```

- **Referencia técnica:** Configura la protección contra ataques CSRF adaptándose a la URL dinámica de Cloud Run.

Paso 4 - Archivos Estáticos en la Nube

- Finalmente, le diremos a Django que mande el CSS y las imágenes al Bucket que creaste, en lugar de guardarlos en el disco local.
 - Ve al final de tu `settings.py` (sección `STATIC_URL`) y agrega esto:

```
# Configuración de Archivos Estáticos (Cloud Storage)
GS_BUCKET_NAME = env("GS_BUCKET_NAME")
STATIC_URL = "/static/"

STORAGES = {
    "default": {
        "BACKEND": "storages.backends.gcloud.GoogleCloudStorage",
    },
    "staticfiles": {
        "BACKEND": "storages.backends.gcloud.GoogleCloudStorage",
    },
}
```

- **Referencia técnica:** Utiliza la librería `django-storages` para conectar el bucket como sistema de archivos

Etapa 7 - Automatización del Despliegue (Cloud Build)

 **Objetivo de la Etapa:** Hasta ahora tenemos todas las piezas sueltas. En esta etapa final, usaremos **Cloud Build** para ensamblarlas. En lugar de subir archivos manualmente por FTP (¡qué antiguo!), crearemos una "Receta de Cocina" (archivo YAML) que le dirá a Google: "Toma mi código, empaquétalo en un contenedor, súbelo al registro, conecta la base de datos, ejecuta las migraciones y despliega la aplicación". Todo con un solo comando.

Paso 1 - La Receta Maestra

- Crea un archivo llamado `cloudmigrate.yaml` en la raíz de tu proyecto (junto a `manage.py`). Este archivo definirá los pasos de construcción y migración.
- Copia y pega este contenido. Este script define 3 pasos críticos: Construir la imagen, Subirla y Crear un "Trabajo" para migrar la base de datos automáticamente (*revisa la siguiente página*):

steps:

1. Construir la imagen del contenedor usando Buildpacks

- id: "Build Container Image"

name: "buildpacksio/pack"

args: ["build", "\${_IMAGE_NAME}", "--builder", "gcr.io/buildpacks/builder"]

2. Subir la imagen al registro de Google (Artifact Registry)

- id: "Push Container Image"

name: "gcr.io/cloud-builders/docker"

args: ["push", "\${_IMAGE_NAME}"]

3. Ejecutar Migraciones de Base de Datos (Crea un Job de Cloud Run)

- id: "Migrate database"

name: "gcr.io/google.com/cloudsdktool/cloud-sdk"

entrypoint: /bin/bash

args:

- "-c"

- |

```
gcloud run jobs create migrate-job \
--region ${_REGION} \
--image ${_IMAGE_NAME} \
--set-cloudsql-instances ${_CLOUD_SQL_CONNECTION_NAME} \
--set-env-vars SETTINGS_NAME=${_SECRET_SETTINGS_NAME} \
--command migrate \
--execute-now || gcloud run jobs update migrate-job \
--region ${_REGION} \
--image ${_IMAGE_NAME} \
--execute-now
```

substitutions:

_INSTANCE_NAME: miapp-db-server # El nombre de tu instancia SQL

_REGION: us-central1 # Tu región elegida

_SERVICE_NAME: polls-service # Nombre de tu servicio web

_SECRET_SETTINGS_NAME: django_settings

_ARTIFACT_REGISTRY: cloud-run-source-deploy

_CLOUD_SQL_CONNECTION_NAME: \${PROJECT_ID}:\${_REGION}:\${_INSTANCE_NAME}

_IMAGE_NAME: \${_REGION}-

docker.pkg.dev/\${PROJECT_ID}/\${_ARTIFACT_REGISTRY}/\${_SERVICE_NAME}

images:

- "\${_IMAGE_NAME}"

Paso 2 - Ejecutar la Construcción (El "Build")

- Abre tu terminal y ejecuta la receta. Google empezará a cocinar tu aplicación. (Nota: Reemplaza `miapp-db-server` si usaste otro nombre para tu SQL).

```
gcloud builds submit --config cloudmigrate.yaml \
  --substitutions _INSTANCE_NAME=miapp-db-server,_REGION=us-central1
```

- Espera unos minutos. Verás logs de construcción. Si termina en "SUCCESS", ¡tu imagen y base de datos están listas!.

Paso 3 - Despliegue del Servicio Web (El "Deploy")

- Ahora que la imagen existe y la base de datos tiene las tablas (gracias a la migración automática), vamos a hacer pública la aplicación.
- Ejecuta este comando para crear el servicio en Cloud Run:

```
gcloud run deploy polls-service \
  --region us-central1 \
  --image us-central1-docker.pkg.dev/PROJECT_ID/cloud-run-source-deploy/polls-service \
  --add-cloudsql-instances PROJECT_ID:us-central1:miapp-db-server \
  --allow-unauthenticated
```

- **Tip:** Reemplaza `PROJECT_ID` con el ID de tu proyecto. La terminal te dará la URL de la imagen exacta en el paso anterior si tienes dudas.

Paso 4 - Corrección de Seguridad (CSRF)

¡Tu app ya vive en internet! Pero si intentas entrar al admin, te dará un error de seguridad (CSRF). Esto pasa porque Django no conoce la URL pública que Google te acaba de asignar. Vamos a decírselo.

- Copia la URL que te dio la terminal (ej: `https://polls-service-xyz.run.app`).
- Ejecuta este comando para actualizar la variable de entorno:

```
gcloud run services update polls-service \
  --region us-central1 \
  --update-env-vars CLOUDRUN_SERVICE_URLS=https://TU-URL-AQUI.run.app
```

- **Referencia técnica:** Inyecta la URL pública para que Django la añada a sus orígenes de confianza.

Paso 5 - Creación del Superusuario (Acceso Admin)

Para poder entrar a `/admin`, necesitas un usuario. Como no tenemos consola interactiva, usaremos un truco rápido con Cloud Run Jobs (similar a la migración).

- Ejecuta este bloque para crear un "Job" que cree al usuario admin automáticamente usando el secreto que configuraste en la **Etapa 5**:
- **Referencia técnica:** Ejecuta `createsuperuser` en modo no interactivo leyendo la password desde Secret Manager.

```
gcloud run jobs create create-superuser \  
--region us-central1 \  
--image us-central1-docker.pkg.dev/PROJECT_ID/cloud-run-source-deploy/polls-service \  
--set-cloudsql-instances PROJECT_ID:us-central1:miapp-db-server \  
--set-env-vars SETTINGS_NAME=django_settings \  
--set-env-vars DJANGO_SUPERUSER_USERNAME=admin \  
--set-env-vars DJANGO_SUPERUSER_EMAIL=admin@example.com \  
--set-secrets DJANGO_SUPERUSER_PASSWORD=superuser_password:latest \  
--command createsuperuser \  
--args "--noinput" \  
--execute-now
```

¡MISIÓN CUMPLIDA! Ve a la URL de tu servicio, agrega `/admin` al final, e inicia sesión con el usuario `admin` y la contraseña que guardaste en tu secreto.

Ahora tienes una app profesional, segura y escalable corriendo en la infraestructura de Google.



Conclusión y Próximos Pasos

¡Felicidades! Has logrado desplegar una aplicación web moderna utilizando una arquitectura Serverless. Lo que has construido hoy va mucho más allá de un simple sitio web; has implementado:

1

Infraestructura como Código

Al definir tus despliegues mediante comandos y configuraciones.

2

Seguridad Gestionada

Al desacoplar tus credenciales del código fuente.

3

Escalabilidad Automática

Tu aplicación en Cloud Run puede escalar desde cero hasta manejar miles de peticiones automáticamente.



Preparación para la Certificación Google Cloud Associate Cloud Engineer

Este laboratorio es una práctica directa para la certificación **Associate Cloud Engineer (ACE)** de Google. Al completar estas etapas, has tocado los pilares fundamentales que evalúa el examen:

- Configuración de un entorno de solución en la nube.
- Planificación y configuración de una solución.
- Despliegue e implementación.
- Garantía de funcionamiento exitoso.

Dominar estos flujos de trabajo te coloca un paso más cerca de obtener una acreditación reconocida globalmente en la industria TI.

¡Éxito!